

```

"""
Benchmark: C-Score Necessity Predictor
=====
Analyzes whether C-Score (contextual analysis) is necessary after
E-Score and Q-Score have masked explicit PII and quasi-identifiers.

Features:
- Checkpoint system for resumable processing
- Incremental saves after each sample
- Graceful interrupt handling (Ctrl+C)
"""

import json
import time
import re
import argparse
import signal
import sys
import hashlib
from dataclasses import dataclass, asdict, field
from pathlib import Path
from typing import List, Dict, Optional, Tuple, Set
from collections import Counter
from datetime import datetime
import numpy as np
import matplotlib.pyplot as plt

# Import the three scoring modules
from e_score import PIIDetector, PIIAnalysisResult
from q_score import QScoreAnalyzer, QScoreResult
from c_score import NarrativeSensitivityDetector, NarrativeAnalysis, RiskLevel

# =====
# High-Risk Entity Combinations (likely to need C-Score)
# =====
HIGH_RISK_COMBOS = [
    {"PERSON", "LOCATION"},
    {"PERSON", "DATE_TIME"},
    {"PERSON", "ORGANIZATION"},
    {"LOCATION", "DATE_TIME"},
    {"PERSON", "LOCATION", "DATE_TIME"},
    {"PERSON", "PHONE_NUMBER"},
    {"PERSON", "EMAIL_ADDRESS"},
]

# Entity types that often appear in sensitive narratives
NARRATIVE_ENTITY_TYPES = {"PERSON", "LOCATION", "ORGANIZATION", "DATE_TIME"}

# High-weight entity types (direct identifiers)
DIRECT_IDENTIFIER_TYPES = {
    "US_SSN", "US_PASSPORT", "US_DRIVER_LICENSE", "CREDIT_CARD",
    "US_BANK_NUMBER", "IBAN_CODE", "EMAIL_ADDRESS", "PHONE_NUMBER"
}

# =====
# Data Classes
# =====
@dataclass
class BenchmarkSample:
    """A single benchmark test case."""
    name: str
    text: str
    expected_critical: bool = False

```

```

def get_hash(self) -> str:
    """Generate a unique hash for this sample based on content."""
    content = f"{self.name}|{self.text}|{self.expected_critical}"
    return hashlib.md5(content.encode()).hexdigest()[:12]

```

```

@dataclass
class PipelineResult:
    """Results from running the full pipeline on a sample."""
    sample_name: str
    sample_hash: str # NEW: For identifying samples across runs
    original_text: str
    expected_critical: bool

    # E-Score results
    e_score: float
    e_score_entity_count: int
    e_score_entity_types: List[str]
    e_score_max_weight: float
    e_score_mean_confidence: float
    e_score_median_confidence: float
    e_score_masked_text: str

    # Q-Score results (run on E-Score masked text)
    q_score: float
    q_score_expected_k: float
    q_score_qi_count: int
    q_score_qi_types: List[str]
    q_score_mean_confidence: float
    q_score_median_confidence: float
    q_score_has_rare_disease: bool
    q_score_has_rare_occupation: bool
    q_score_masked_text: str

    # C-Score results (run on fully masked text)
    c_score_risk_level: str
    c_score_risk_score: float
    c_score_factors_detected: List[str]
    c_score_factor_count: int
    c_score_deemed_sensitive: bool

    # Alternative Features
    text_length: int
    word_count: int
    sentence_count: int
    masked_text_length: int
    mask_token_count: int
    mask_ratio: float
    chars_masked_ratio: float
    remaining_word_count: int
    total_entity_count: int
    unique_entity_type_count: int
    entity_density: float
    has_person: bool
    has_location: bool
    has_datetime: bool
    has_organization: bool
    has_direct_identifier: bool
    narrative_entity_count: int
    direct_identifier_count: int
    has_high_risk_combo: bool
    high_risk_combo_count: int
    entity_type_combo_signature: str

```

```

e_q_score_sum: float
e_q_score_product: float
e_q_score_max: float
score_disparity: float
e_score_coverage: float
q_score_coverage: float
combined_mean_confidence: float
combined_median_confidence: float

# Timing
e_score_time: float
q_score_time: float
c_score_time: float

# Metadata
processed_at: str = "" # ISO timestamp

```

```

@dataclass
class CheckpointData:
    """Data structure for checkpoint files."""
    input_file_hash: str
    c_score_threshold: float
    total_samples: int
    processed_count: int
    results: List[Dict] # Serialized PipelineResults
    last_updated: str
    version: str = "2.0" # For compatibility checking

    def to_dict(self) -> Dict:
        return asdict(self)

    @classmethod
    def from_dict(cls, data: Dict) -> 'CheckpointData':
        return cls(**data)

```

```

@dataclass
class BenchmarkAnalysis:
    """Analysis of benchmark results to determine C-Score necessity."""
    total_samples: int
    c_score_sensitive_count: int
    c_score_not_sensitive_count: int
    correlations: Dict[str, float]
    feature_importance: List[Tuple[str, float]]
    suggested_e_score_threshold: float
    suggested_q_score_threshold: float
    suggested_combined_threshold: float
    best_predictors: List[str]

```

```

# =====
# Checkpoint Manager
# =====
class CheckpointManager:
    """Manages saving and loading of benchmark progress."""

    def __init__(self, output_dir: Path, input_file: str, c_score_threshold:
float):
        self.output_dir = output_dir
        self.input_file = input_file
        self.c_score_threshold = c_score_threshold
        self.checkpoint_file = output_dir / "checkpoint.json"
        self.backup_file = output_dir / "checkpoint.backup.json"

```

```

# Calculate input file hash for validation
self.input_hash = self._hash_input_file()

# Results storage
self.results: Dict[str, PipelineResult] = {} # hash -> result
self.processed_hashes: Set[str] = set()

# Interrupt handling
self._interrupted = False
self._setup_signal_handlers()

def _hash_input_file(self) -> str:
    """Generate hash of input file for change detection."""
    with open(self.input_file, 'rb') as f:
        return hashlib.md5(f.read()).hexdigest()

def _setup_signal_handlers(self):
    """Setup graceful interrupt handling."""
    signal.signal(signal.SIGINT, self._handle_interrupt)
    signal.signal(signal.SIGTERM, self._handle_interrupt)

def _handle_interrupt(self, signum, frame):
    """Handle Ctrl+C gracefully."""
    if self._interrupted:
        print("\n\nForce quit. Progress was saved at last checkpoint.")
        sys.exit(1)

    self._interrupted = True
    print("\n\n" + "=" * 70)
    print("INTERRUPT RECEIVED - Saving checkpoint...")
    print("Press Ctrl+C again to force quit")
    print("=" * 70)

@property
def was_interrupted(self) -> bool:
    return self._interrupted

def load_checkpoint(self) -> Tuple[bool, int]:
    """
    Load existing checkpoint if valid.
    Returns: (checkpoint_loaded, num_results_loaded)
    """
    if not self.checkpoint_file.exists():
        print("No checkpoint found. Starting fresh.")
        return False, 0

    try:
        with open(self.checkpoint_file, 'r') as f:
            data = json.load(f)

        checkpoint = CheckpointData.from_dict(data)

        # Validate checkpoint
        if checkpoint.input_file_hash != self.input_hash:
            print(f"â§ i, Input file has changed since last run.")
            print("Checkpoint invalidated. Starting fresh.")
            self._backup_old_checkpoint()
            return False, 0

        if checkpoint.c_score_threshold != self.c_score_threshold:
            print(f"â§ i, C-Score threshold changed:
{checkpoint.c_score_threshold} -> {self.c_score_threshold}")
            print("Checkpoint invalidated. Starting fresh.")

```

```

        self._backup_old_checkpoint()
        return False, 0

    if checkpoint.version != "2.0":
        print(f"â§ i, Checkpoint version mismatch:
{checkpoint.version}")
        print(" Checkpoint invalidated. Starting fresh.")
        self._backup_old_checkpoint()
        return False, 0

    # Load results
    for result_dict in checkpoint.results:
        result = self._dict_to_result(result_dict)
        self.results[result.sample_hash] = result
        self.processed_hashes.add(result.sample_hash)

    print(f"â€“ Loaded checkpoint:
{len(self.results)}/{checkpoint.total_samples} samples processed")
    print(f" Last updated: {checkpoint.last_updated}")

    return True, len(self.results)

except Exception as e:
    print(f"â§ i, Error loading checkpoint: {e}")
    print(" Starting fresh.")
    self._backup_old_checkpoint()
    return False, 0

def _backup_old_checkpoint(self):
    """Backup old checkpoint before overwriting."""
    if self.checkpoint_file.exists():
        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        backup_name = f"checkpoint.{timestamp}.backup.json"
        backup_path = self.output_dir / backup_name
        self.checkpoint_file.rename(backup_path)
        print(f" Old checkpoint backed up to: {backup_name}")

def save_checkpoint(self, total_samples: int):
    """Save current progress to checkpoint file."""
    checkpoint = CheckpointData(
        input_file_hash=self.input_hash,
        c_score_threshold=self.c_score_threshold,
        total_samples=total_samples,
        processed_count=len(self.results),
        results=[asdict(r) for r in self.results.values()],
        last_updated=datetime.now().isoformat()
    )

    # Write to temp file first, then rename (atomic on most filesystems)
    temp_file = self.checkpoint_file.with_suffix('.tmp')
    with open(temp_file, 'w') as f:
        json.dump(checkpoint.to_dict(), f)
    temp_file.rename(self.checkpoint_file)

def add_result(self, result: PipelineResult, total_samples: int, auto_save:
bool = True):
    """Add a result and optionally save checkpoint."""
    self.results[result.sample_hash] = result
    self.processed_hashes.add(result.sample_hash)

    if auto_save:
        self.save_checkpoint(total_samples)

def is_processed(self, sample: BenchmarkSample) -> bool:

```

```

        """Check if a sample has already been processed."""
        return sample.get_hash() in self.processed_hashes

    def get_result(self, sample: BenchmarkSample) -> Optional[PipelineResult]:
        """Get existing result for a sample."""
        return self.results.get(sample.get_hash())

    def get_all_results(self) -> List[PipelineResult]:
        """Get all results in a list."""
        return list(self.results.values())

    def clear_checkpoint(self):
        """Clear checkpoint file (for fresh start)."""
        if self.checkpoint_file.exists():
            self._backup_old_checkpoint()
        self.results.clear()
        self.processed_hashes.clear()

    def _dict_to_result(self, d: Dict) -> PipelineResult:
        """Convert dictionary back to PipelineResult."""
        return PipelineResult(**d)

# =====
# Input File Parser
# =====
def parse_input_file(filepath: str, max_samples: Optional[int] = None) ->
List[BenchmarkSample]:
    """Parse the input text file containing benchmark samples."""
    samples = []

    with open(filepath, 'r', encoding='utf-8') as f:
        content = f.read()

    raw_samples = content.split('---')

    for raw in raw_samples:
        raw = raw.strip()
        if not raw:
            continue

        name_match = re.search(r'#\s*name:\s*(.+)', raw, re.IGNORECASE)
        critical_match = re.search(r'#\s*expected_critical:\s*(true|false)',
raw, re.IGNORECASE)

        name = name_match.group(1).strip() if name_match else
f"Sample_{len(samples) + 1}"
        expected_critical = critical_match.group(1).lower() == 'true' if
critical_match else False

        text_lines = [line for line in raw.split('\n') if not
line.strip().startswith('#')]
        text = '\n'.join(text_lines).strip()

        if text:
            samples.append(BenchmarkSample(name=name, text=text,
expected_critical=expected_critical))

        if max_samples and len(samples) >= max_samples:
            break

    return samples

```

```

# =====
# Helper Functions
# =====
def count_sentences(text: str) -> int:
    """Rough sentence count based on punctuation."""
    return len(re.findall(r'[.!?]+' , text)) or 1

def check_high_risk_combos(entity_types: Set[str]) -> Tuple[bool, int]:
    """Check if entity types contain high-risk combinations."""
    count = 0
    for combo in HIGH_RISK_COMBOS:
        if combo.issubset(entity_types):
            count += 1
    return count > 0, count

def calculate_coverage(original: str, masked: str) -> float:
    """Calculate what fraction of characters were masked."""
    if len(original) == 0:
        return 0.0
    mask_tokens = re.findall(r'\[[A-Z_]+\]', masked)
    estimated_original_len = len(masked) - sum(len(t) for t in mask_tokens)
    chars_removed = len(original) - estimated_original_len
    return max(0, chars_removed) / len(original)

# =====
# Pipeline Runner
# =====
class BenchmarkPipeline:
    """Runs the E-Score -> Q-Score -> C-Score pipeline with checkpointing."""

    def __init__(
        self,
        checkpoint_manager: CheckpointManager,
        c_score_sensitivity_threshold: float = 0.3
    ):
        self.checkpoint = checkpoint_manager
        self.c_score_sensitivity_threshold = c_score_sensitivity_threshold

        # Lazy initialization of detectors
        self._e_detector = None
        self._q_analyzer = None
        self._c_detector = None

    @property
    def e_detector(self):
        if self._e_detector is None:
            print("Initializing E-Score detector...")
            self._e_detector = PIIDetector()
        return self._e_detector

    @property
    def q_analyzer(self):
        if self._q_analyzer is None:
            print("Initializing Q-Score analyzer...")
            self._q_analyzer = QScoreAnalyzer()
        return self._q_analyzer

    @property
    def c_detector(self):
        if self._c_detector is None:
            print("Initializing C-Score detector...")

```

```

        self._c_detector = NarrativeSensitivityDetector()
    return self._c_detector

def run_sample(self, sample: BenchmarkSample) -> PipelineResult:
    """Run the full pipeline on a single sample."""

    original_text = sample.text
    sample_hash = sample.get_hash()

    #
    =====
    # Step 1: E-Score
    #
    =====
    start = time.time()
    e_result = self.e_detector.analyze(original_text)
    e_time = time.time() - start

    e_masked = e_result.masked_text
    e_entity_types = list(set(e.entity_type for e in e_result.entities))
    e_entity_types_set = set(e_entity_types)
    e_max_weight = max((e.weight for e in e_result.entities), default=0)

    e_confidences = [e.confidence for e in e_result.entities]
    e_mean_conf = np.mean(e_confidences) if e_confidences else 0.0
    e_median_conf = np.median(e_confidences) if e_confidences else 0.0

    #
    =====
    # Step 2: Q-Score (on E-masked text)
    #
    =====
    start = time.time()
    q_report = self.q_analyzer.get_report(e_masked)
    q_time = time.time() - start

    q_masked = q_report.get("masked_text", e_masked)
    q_qi_types = [qi["type"] for qi in
q_report.get("detected_quasi_identifiers", [])]
    q_qi_types_set = set(q_qi_types)

    q_confidences = [
        qi.get("confidence", 0)
        for qi in q_report.get("detected_quasi_identifiers", [])
    ]
    q_mean_conf = np.mean(q_confidences) if q_confidences else 0.0
    q_median_conf = np.median(q_confidences) if q_confidences else 0.0

    has_rare_disease = any(
        qi["type"] == "disease" and
        qi.get("frequency", {}).get("probability", 1) < 0.001
        for qi in q_report.get("detected_quasi_identifiers", [])
    )
    has_rare_occupation = any(
        qi["type"] == "occupation" and
        qi.get("frequency", {}).get("probability", 1) < 0.001
        for qi in q_report.get("detected_quasi_identifiers", [])
    )

    #
    =====
    # Step 3: C-Score (on fully masked text)
    #
    =====

```



```

start = time.time()
try:
    c_result = self.c_detector.analyze(q_masked)
    c_time = time.time() - start

    c_factors_detected = [f.name for f in c_result.factors if
f.detected]
    c_risk_level = c_result.overall_risk.value
    c_risk_score = c_result.risk_score
except Exception as e:
    print(f" C-Score error for {sample.name}: {e}")
    c_time = 0
    c_factors_detected = []
    c_risk_level = "ERROR"
    c_risk_score = 0

c_deemed_sensitive = c_risk_score >= self.c_score_sensitivity_threshold

#
=====
# Calculate Features
#
=====
text_length = len(original_text)
word_count = len(original_text.split())
sentence_count = count_sentences(original_text)

masked_text_length = len(q_masked)
mask_tokens = re.findall(r'\[[A-Z_]+\]', q_masked)
mask_token_count = len(mask_tokens)
mask_ratio = mask_token_count / max(word_count, 1)
chars_masked_ratio = 1 - (masked_text_length / max(text_length, 1))
remaining_word_count = len(q_masked.split())

e_entity_count = len(e_result.entities)
q_qi_count = len(q_report.get("detected_quasi_identifiers", []))
total_entity_count = e_entity_count + q_qi_count

all_entity_types = e_entity_types_set | q_qi_types_set
unique_entity_type_count = len(all_entity_types)
entity_density = (total_entity_count / max(word_count, 1)) * 100

has_person = "PERSON" in e_entity_types_set
has_location = "LOCATION" in e_entity_types_set
has_datetime = "DATE_TIME" in e_entity_types_set
has_organization = "ORGANIZATION" in e_entity_types_set
has_direct_idenfifier = bool(e_entity_types_set &
DIRECT_IDENTIFIER_TYPES)

narrative_entity_count = sum(
    1 for e in e_result.entities
    if e.entity_type in NARRATIVE_ENTITY_TYPES
)
direct_idenfifier_count = sum(
    1 for e in e_result.entities
    if e.entity_type in DIRECT_IDENTIFIER_TYPES
)

has_high_risk_combo, high_risk_combo_count =
check_high_risk_combos(e_entity_types_set)
entity_type_combo_signature = "|".join(sorted(all_entity_types))

e_score_val = e_result.e_score
q_score_val = q_report["q_score"]

```

```

e_q_score_sum = e_score_val + q_score_val
e_q_score_product = e_score_val * q_score_val
e_q_score_max = max(e_score_val, q_score_val)
score_disparity = abs(e_score_val - q_score_val)

e_score_coverage = calculate_coverage(original_text, e_masked)
q_score_coverage = calculate_coverage(e_masked, q_masked)

all_confidences = e_confidences + q_confidences
combined_mean = np.mean(all_confidences) if all_confidences else 0.0
combined_median = np.median(all_confidences) if all_confidences else 0.0

return PipelineResult(
    sample_name=sample.name,
    sample_hash=sample_hash,
    original_text=original_text,
    expected_critical=sample.expected_critical,

    # E-Score
    e_score=e_score_val,
    e_score_entity_count=e_entity_count,
    e_score_entity_types=e_entity_types,
    e_score_max_weight=e_max_weight,
    e_score_mean_confidence=float(e_mean_conf),
    e_score_median_confidence=float(e_median_conf),
    e_score_masked_text=e_masked,

    # Q-Score
    q_score=q_score_val,
    q_score_expected_k=q_report["expected_k"],
    q_score_qi_count=q_qi_count,
    q_score_qi_types=q_qi_types,
    q_score_mean_confidence=float(q_mean_conf),
    q_score_median_confidence=float(q_median_conf),
    q_score_has_rare_disease=has_rare_disease,
    q_score_has_rare_occupation=has_rare_occupation,
    q_score_masked_text=q_masked,

    # C-Score
    c_score_risk_level=c_risk_level,
    c_score_risk_score=c_risk_score,
    c_score_factors_detected=c_factors_detected,
    c_score_factor_count=len(c_factors_detected),
    c_score_deemed_sensitive=c_deemed_sensitive,

    # Features
    text_length=text_length,
    word_count=word_count,
    sentence_count=sentence_count,
    masked_text_length=masked_text_length,
    mask_token_count=mask_token_count,
    mask_ratio=mask_ratio,
    chars_masked_ratio=chars_masked_ratio,
    remaining_word_count=remaining_word_count,
    total_entity_count=total_entity_count,
    unique_entity_type_count=unique_entity_type_count,
    entity_density=entity_density,
    has_person=has_person,
    has_location=has_location,
    has_datetime=has_datetime,
    has_organization=has_organization,
    has_direct_identifier=has_direct_identifier,
    narrative_entity_count=narrative_entity_count,
    direct_identifier_count=direct_identifier_count,

```

```

        has_high_risk_combo=has_high_risk_combo,
        high_risk_combo_count=high_risk_combo_count,
        entity_type_combo_signature=entity_type_combo_signature,
        e_q_score_sum=e_q_score_sum,
        e_q_score_product=e_q_score_product,
        e_q_score_max=e_q_score_max,
        score_disparity=score_disparity,
        e_score_coverage=e_score_coverage,
        q_score_coverage=q_score_coverage,
        combined_mean_confidence=float(combined_mean),
        combined_median_confidence=float(combined_median),

        # Timing
        e_score_time=e_time,
        q_score_time=q_time,
        c_score_time=c_time,

        # Metadata
        processed_at=datetime.now().isoformat()
    )

    def run_all(self, samples: List[BenchmarkSample], save_interval: int = 1) ->
List[PipelineResult]:
        """
        Run pipeline on all samples with checkpointing.

        Args:
            samples: List of samples to process
            save_interval: Save checkpoint every N samples (default: 1 for
maximum safety)
        """
        total = len(samples)
        processed_new = 0
        skipped = 0

        for i, sample in enumerate(samples):
            # Check for interrupt
            if self.checkpoint.was_interrupted:
                print(f"\nStopping at sample {i}/{total} due to interrupt.")
                break

            # Check if already processed
            if self.checkpoint.is_processed(sample):
                skipped += 1
                if skipped <= 5 or skipped % 10 == 0:
                    print(f" [{i+1}/{total}] {sample.name} - CACHED ⚡")
                continue

            # Process sample
            print(f" [{i+1}/{total}] Processing: {sample.name}...", end=" ",
flush=True)

            try:
                start_time = time.time()
                result = self.run_sample(sample)
                elapsed = time.time() - start_time

                # Save result with checkpoint
                auto_save = (processed_new + 1) % save_interval == 0
                self.checkpoint.add_result(result, total, auto_save=auto_save)

                processed_new += 1
                status = "ðŸŒˆ SENS" if result.c_score_deemed_sensitive else
"ðŸŒˆ OK"

```

```

        print(f"{status} ({elapsed:.1f}s)")

    except Exception as e:
        print(f"ERROR: {e}")
        # Continue to next sample

    # Final save
    self.checkpoint.save_checkpoint(total)

    # Summary
    print()
    print(f"Processing complete:")
    print(f"  - New samples processed: {processed_new}")
    print(f"  - Cached samples skipped: {skipped}")
    print(f"  - Total results: {len(self.checkpoint.results)}")

    if self.checkpoint.was_interrupted:
        print(f"\nâ§ i, Run was interrupted. Resume by running the same
command again.")

    return self.checkpoint.get_all_results()

# =====
# Visualization Functions
# =====
def create_visualizations(results: List[PipelineResult], output_dir: Path):
    """Create visualizations for alternative features."""

    if not results:
        print("No results to visualize.")
        return

    sensitive = [r for r in results if r.c_score_deemed_sensitive]
    not_sensitive = [r for r in results if not r.c_score_deemed_sensitive]

    sensitive_color = '#e74c3c'
    not_sensitive_color = '#2ecc71'

    # =====
    # Figure 1: Alternative Features Scatter Plots (2x3)
    # =====
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    fig.suptitle('Alternative Features vs C-Score Sensitivity', fontsize=14,
fontweight='bold')

    # Plot 1: Entity Count
    ax = axes[0, 0]
    if sensitive:
        ax.scatter([r.total_entity_count for r in sensitive],
[r.c_score_risk_score for r in sensitive],
c=sensitive_color, label='Sensitive', alpha=0.7, s=60,
edgecolors='black')
    if not_sensitive:
        ax.scatter([r.total_entity_count for r in not_sensitive],
[r.c_score_risk_score for r in not_sensitive],
c=not_sensitive_color, label='Not Sensitive', alpha=0.7,
s=60, edgecolors='black')
    ax.axhline(y=0.3, color='gray', linestyle='--', alpha=0.5)
    ax.set_xlabel('Total Entity Count')
    ax.set_ylabel('C-Score Risk Score')
    ax.set_title('Total Entity Count')
    ax.legend(loc='best', fontsize=8)
    ax.grid(True, alpha=0.3)

```

```

# Plot 2: Entity Density
ax = axes[0, 1]
if sensitive:
    ax.scatter([r.entity_density for r in sensitive],
               [r.c_score_risk_score for r in sensitive],
               c=sensitive_color, alpha=0.7, s=60, edgecolors='black')
if not_sensitive:
    ax.scatter([r.entity_density for r in not_sensitive],
               [r.c_score_risk_score for r in not_sensitive],
               c=not_sensitive_color, alpha=0.7, s=60, edgecolors='black')
ax.axhline(y=0.3, color='gray', linestyle='--', alpha=0.5)
ax.set_xlabel('Entity Density (per 100 words)')
ax.set_ylabel('C-Score Risk Score')
ax.set_title('Entity Density')
ax.grid(True, alpha=0.3)

# Plot 3: Mask Ratio
ax = axes[0, 2]
if sensitive:
    ax.scatter([r.mask_ratio for r in sensitive],
               [r.c_score_risk_score for r in sensitive],
               c=sensitive_color, alpha=0.7, s=60, edgecolors='black')
if not_sensitive:
    ax.scatter([r.mask_ratio for r in not_sensitive],
               [r.c_score_risk_score for r in not_sensitive],
               c=not_sensitive_color, alpha=0.7, s=60, edgecolors='black')
ax.axhline(y=0.3, color='gray', linestyle='--', alpha=0.5)
ax.set_xlabel('Mask Ratio (tokens/words)')
ax.set_ylabel('C-Score Risk Score')
ax.set_title('Mask Ratio')
ax.grid(True, alpha=0.3)

# Plot 4: Unique Entity Types
ax = axes[1, 0]
if sensitive:
    ax.scatter([r.unique_entity_type_count for r in sensitive],
               [r.c_score_risk_score for r in sensitive],
               c=sensitive_color, alpha=0.7, s=60, edgecolors='black')
if not_sensitive:
    ax.scatter([r.unique_entity_type_count for r in not_sensitive],
               [r.c_score_risk_score for r in not_sensitive],
               c=not_sensitive_color, alpha=0.7, s=60, edgecolors='black')
ax.axhline(y=0.3, color='gray', linestyle='--', alpha=0.5)
ax.set_xlabel('Unique Entity Type Count')
ax.set_ylabel('C-Score Risk Score')
ax.set_title('Unique Entity Types')
ax.grid(True, alpha=0.3)

# Plot 5: Narrative Entity Count
ax = axes[1, 1]
if sensitive:
    ax.scatter([r.narrative_entity_count for r in sensitive],
               [r.c_score_risk_score for r in sensitive],
               c=sensitive_color, alpha=0.7, s=60, edgecolors='black')
if not_sensitive:
    ax.scatter([r.narrative_entity_count for r in not_sensitive],
               [r.c_score_risk_score for r in not_sensitive],
               c=not_sensitive_color, alpha=0.7, s=60, edgecolors='black')
ax.axhline(y=0.3, color='gray', linestyle='--', alpha=0.5)
ax.set_xlabel('Narrative Entity Count (PERSON+LOC+ORG+DATE)')
ax.set_ylabel('C-Score Risk Score')
ax.set_title('Narrative Entities')
ax.grid(True, alpha=0.3)

```

```

# Plot 6: E+Q Score Sum
ax = axes[1, 2]
if sensitive:
    ax.scatter([r.e_q_score_sum for r in sensitive],
               [r.c_score_risk_score for r in sensitive],
               c=sensitive_color, alpha=0.7, s=60, edgecolors='black')
if not_sensitive:
    ax.scatter([r.e_q_score_sum for r in not_sensitive],
               [r.c_score_risk_score for r in not_sensitive],
               c=not_sensitive_color, alpha=0.7, s=60, edgecolors='black')
ax.axhline(y=0.3, color='gray', linestyle='--', alpha=0.5)
ax.set_xlabel('E-Score + Q-Score Sum')
ax.set_ylabel('C-Score Risk Score')
ax.set_title('Combined Score Sum')
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(output_dir / 'alternative_features_scatter.png', dpi=150,
bbox_inches='tight')
plt.close()

# =====
# Figure 2: Boolean Features Bar Chart
# =====
fig, ax = plt.subplots(figsize=(12, 6))

bool_features = [
    ('has_person', 'Has PERSON'),
    ('has_location', 'Has LOCATION'),
    ('has_datetime', 'Has DATE_TIME'),
    ('has_organization', 'Has ORG'),
    ('has_direct_identifier', 'Has Direct ID'),
    ('has_high_risk_combo', 'Has Risk Combo'),
]

x = np.arange(len(bool_features))
width = 0.35

sens_rates = []
not_sens_rates = []

for attr, _ in bool_features:
    sens_with = sum(1 for r in sensitive if getattr(r, attr))
    sens_rate = sens_with / len(sensitive) if sensitive else 0
    sens_rates.append(sens_rate)

    not_sens_with = sum(1 for r in not_sensitive if getattr(r, attr))
    not_sens_rate = not_sens_with / len(not_sensitive) if not_sensitive else 0
    not_sens_rates.append(not_sens_rate)

bars1 = ax.bar(x - width/2, not_sens_rates, width, label='Not Sensitive',
               color=not_sensitive_color, edgecolor='black')
bars2 = ax.bar(x + width/2, sens_rates, width, label='Sensitive',
               color=sensitive_color, edgecolor='black')

ax.set_ylabel('Proportion with Feature')
ax.set_title('Boolean Feature Presence by Sensitivity')
ax.set_xticks(x)
ax.set_xticklabels([f[1] for f in bool_features])
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
ax.set_ylim(0, 1.1)

```

```

for bar in bars1 + bars2:
    height = bar.get_height()
    ax.annotate(f'{height:.0%}',
                xy=(bar.get_x() + bar.get_width() / 2, height),
                xytext=(0, 3), textcoords="offset points",
                ha='center', va='bottom', fontsize=8)

plt.tight_layout()
plt.savefig(output_dir / 'boolean_features_bars.png', dpi=150,
bbox_inches='tight')
plt.close()

# =====
# Figure 3: Box Plots for Numeric Features
# =====
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
fig.suptitle('Feature Distributions by C-Score Sensitivity', fontsize=14,
fontweight='bold')

numeric_features = [
    ('total_entity_count', 'Total Entity Count'),
    ('entity_density', 'Entity Density'),
    ('mask_ratio', 'Mask Ratio'),
    ('unique_entity_type_count', 'Unique Entity Types'),
    ('narrative_entity_count', 'Narrative Entities'),
    ('e_q_score_sum', 'E+Q Score Sum'),
]

for idx, (attr, title) in enumerate(numeric_features):
    ax = axes[idx // 3, idx % 3]

    data = [
        [getattr(r, attr) for r in not_sensitive] or [0],
        [getattr(r, attr) for r in sensitive] or [0]
    ]

    bp = ax.boxplot(data, tick_labels=['Not Sensitive', 'Sensitive'],
patch_artist=True)
    bp['boxes'][0].set_facecolor(not_sensitive_color)
    bp['boxes'][1].set_facecolor(sensitive_color)
    ax.set_title(title)
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(output_dir / 'feature_distributions_boxplot.png', dpi=150,
bbox_inches='tight')
plt.close()

# =====
# Figure 4: Progress/Timeline Chart (NEW)
# =====
fig, ax = plt.subplots(figsize=(12, 6))

# Sort by processing time if available
sorted_results = sorted(results, key=lambda r: r.processed_at or "")

cumulative_sensitive = []
cumulative_total = []
running_sens = 0

for i, r in enumerate(sorted_results, 1):
    if r.c_score_deemed_sensitive:
        running_sens += 1

```

```

        cumulative_sensitive.append(running_sens)
        cumulative_total.append(i)

    ax.plot(cumulative_total, cumulative_sensitive, 'r-', linewidth=2,
label='Cumulative Sensitive')
    ax.fill_between(cumulative_total, 0, cumulative_sensitive, alpha=0.3,
color='red')
    ax.plot(cumulative_total, cumulative_total, 'g--', linewidth=1, alpha=0.5,
label='Total Samples')

    ax.set_xlabel('Samples Processed')
    ax.set_ylabel('Count')
    ax.set_title('Cumulative Sensitivity Detection Over Processing')
    ax.legend()
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(output_dir / 'processing_progress.png', dpi=150,
bbox_inches='tight')
plt.close()

# =====
# Figure 5: Correlation Heatmap
# =====
numeric_attrs = [
    'total_entity_count', 'entity_density', 'mask_ratio',
    'unique_entity_type_count', 'narrative_entity_count',
    'e_q_score_sum', 'e_score', 'q_score',
    'combined_mean_confidence', 'c_score_risk_score'
]

data_matrix = np.array([
    [getattr(r, attr) for r in results]
    for attr in numeric_attrs
])

corr_matrix = np.corrcoef(data_matrix)

fig, ax = plt.subplots(figsize=(12, 10))
im = ax.imshow(corr_matrix, cmap='RdYlGn', vmin=-1, vmax=1)

ax.set_xticks(np.arange(len(numeric_attrs)))
ax.set_yticks(np.arange(len(numeric_attrs)))
ax.set_xticklabels([a.replace('_', '\n') for a in numeric_attrs],
fontsize=8)
ax.set_yticklabels([a.replace('_', '\n') for a in numeric_attrs],
fontsize=8)

plt.setp(ax.get_xticklabels(), rotation=45, ha="right",
rotation_mode="anchor")

for i in range(len(numeric_attrs)):
    for j in range(len(numeric_attrs)):
        text = ax.text(j, i, f'{corr_matrix[i, j]:.2f}',
                        ha="center", va="center", color="black", fontsize=7)

ax.set_title('Feature Correlation Matrix', fontsize=14)
fig.colorbar(im, ax=ax, shrink=0.8)

plt.tight_layout()
plt.savefig(output_dir / 'correlation_heatmap.png', dpi=150,
bbox_inches='tight')
plt.close()

```



```

print(f"Saved visualizations to {output_dir}")

# =====
# Analysis Functions
# =====
def analyze_results(results: List[PipelineResult]) -> BenchmarkAnalysis:
    """Analyze benchmark results with expanded feature set."""

    if not results:
        return BenchmarkAnalysis(
            total_samples=0,
            c_score_sensitive_count=0,
            c_score_not_sensitive_count=0,
            correlations={},
            feature_importance=[],
            suggested_e_score_threshold=0.5,
            suggested_q_score_threshold=0.5,
            suggested_combined_threshold=1.0,
            best_predictors=[]
        )

    sensitive_count = sum(1 for r in results if r.c_score_deemed_sensitive)
    not_sensitive_count = len(results) - sensitive_count

    c_sensitive = np.array([r.c_score_deemed_sensitive for r in
results]).astype(float)

    def safe_corr(x, y):
        if len(x) < 2 or np.std(x) == 0 or np.std(y) == 0:
            return 0.0
        return float(np.corrcoef(x, y)[0, 1])

    feature_extractors = {
        "e_mean_confidence": lambda r: r.e_score_mean_confidence,
        "e_median_confidence": lambda r: r.e_score_median_confidence,
        "q_mean_confidence": lambda r: r.q_score_mean_confidence,
        "q_median_confidence": lambda r: r.q_score_median_confidence,
        "combined_mean_confidence": lambda r: r.combined_mean_confidence,
        "combined_median_confidence": lambda r: r.combined_median_confidence,
        "e_score": lambda r: r.e_score,
        "q_score": lambda r: r.q_score,
        "e_q_score_sum": lambda r: r.e_q_score_sum,
        "e_q_score_product": lambda r: r.e_q_score_product,
        "e_q_score_max": lambda r: r.e_q_score_max,
        "score_disparity": lambda r: r.score_disparity,
        "total_entity_count": lambda r: r.total_entity_count,
        "e_score_entity_count": lambda r: r.e_score_entity_count,
        "q_score_qi_count": lambda r: r.q_score_qi_count,
        "unique_entity_type_count": lambda r: r.unique_entity_type_count,
        "entity_density": lambda r: r.entity_density,
        "narrative_entity_count": lambda r: r.narrative_entity_count,
        "direct_identifier_count": lambda r: r.direct_identifier_count,
        "mask_ratio": lambda r: r.mask_ratio,
        "mask_token_count": lambda r: r.mask_token_count,
        "chars_masked_ratio": lambda r: r.chars_masked_ratio,
        "e_score_coverage": lambda r: r.e_score_coverage,
        "q_score_coverage": lambda r: r.q_score_coverage,
        "text_length": lambda r: r.text_length,
        "word_count": lambda r: r.word_count,
        "sentence_count": lambda r: r.sentence_count,
        "has_person": lambda r: float(r.has_person),
        "has_location": lambda r: float(r.has_location),
        "has_datetime": lambda r: float(r.has_datetime),
    }

```

```

        "has_organization": lambda r: float(r.has_organization),
        "has_direct_identifier": lambda r: float(r.has_direct_identifier),
        "has_high_risk_combo": lambda r: float(r.has_high_risk_combo),
        "high_risk_combo_count": lambda r: r.high_risk_combo_count,
    }

    correlations = {}
    for name, extractor in feature_extractors.items():
        values = np.array([extractor(r) for r in results])
        correlations[f"{name}_vs_c_sensitive"] = safe_corr(values, c_sensitive)

    feature_importance = sorted(
        [(name.replace("_vs_c_sensitive", ""), abs(corr))
         for name, corr in correlations.items()],
        key=lambda x: x[1],
        reverse=True
    )

    best_predictors = [name for name, corr in feature_importance if corr > 0.3]

    e_scores = np.array([r.e_score for r in results])
    q_scores = np.array([r.q_score for r in results])

    e_threshold = find_threshold(e_scores, c_sensitive)
    q_threshold = find_threshold(q_scores, c_sensitive)
    combined_threshold = find_threshold(e_scores + q_scores, c_sensitive)

    return BenchmarkAnalysis(
        total_samples=len(results),
        c_score_sensitive_count=sensitive_count,
        c_score_not_sensitive_count=not_sensitive_count,
        correlations=correlations,
        feature_importance=feature_importance,
        suggested_e_score_threshold=e_threshold,
        suggested_q_score_threshold=q_threshold,
        suggested_combined_threshold=combined_threshold,
        best_predictors=best_predictors
    )

def find_threshold(scores: np.ndarray, c_sensitive: np.ndarray) -> float:
    """Find threshold that best separates sensitive/not sensitive cases."""
    if len(scores) < 2:
        return 0.5

    best_threshold = 0.5
    best_separation = -float('inf')

    for threshold in np.linspace(np.min(scores), np.max(scores), 20):
        predicted_safe = scores < threshold
        true_negatives = np.sum((~c_sensitive.astype(bool)) & predicted_safe)
        false_negatives = np.sum(c_sensitive.astype(bool) & predicted_safe)
        separation = true_negatives - 2 * false_negatives

        if separation > best_separation:
            best_separation = separation
            best_threshold = threshold

    return float(best_threshold)

# =====
# Main Benchmark Runner
# =====

```

```

def run_benchmark(
    input_file: str,
    output_dir: str = "benchmark_outputs",
    max_samples: Optional[int] = None,
    c_score_threshold: float = 0.3,
    fresh_start: bool = False
):
    """
    Run the full benchmark suite with checkpoint support.

    Args:
        input_file: Path to input samples file
        output_dir: Output directory for results
        max_samples: Limit number of samples (None = all)
        c_score_threshold: Threshold for C-Score sensitivity
        fresh_start: If True, ignore existing checkpoint
    """

    print("=" * 70)
    print("C-SCORE NECESSITY BENCHMARK (with Checkpointing)")
    print("=" * 70)
    print()

    out_path = Path(output_dir)
    out_path.mkdir(exist_ok=True)

    print(f"Loading samples from: {input_file}")
    samples = parse_input_file(input_file, max_samples)
    print(f"Loaded {len(samples)} benchmark samples")

    if max_samples:
        print(f"(Limited to {max_samples} samples)")
        print()

    # Initialize checkpoint manager
    checkpoint = CheckpointManager(out_path, input_file, c_score_threshold)

    # Handle fresh start
    if fresh_start:
        print("Fresh start requested - clearing checkpoint...")
        checkpoint.clear_checkpoint()
    else:
        checkpoint.load_checkpoint()

    # Initialize pipeline with checkpoint
    pipeline = BenchmarkPipeline(checkpoint,
c_score_sensitivity_threshold=c_score_threshold)

    print("\n" + "-" * 70)
    print("PROCESSING SAMPLES")
    print("-" * 70)

    results = pipeline.run_all(samples)

    # Save raw results
    results_data = [asdict(r) for r in results]
    with open(out_path / "pipeline_results.json", "w") as f:
        json.dump(results_data, f, indent=2, default=str)
    print(f"\nSaved pipeline results to {out_path / 'pipeline_results.json'}")

    # Only generate visualizations and analysis if we have results and weren't
    interrupted
    if results and not checkpoint.was_interrupted:
        print("\nGenerating visualizations...")

```

```

create_visualizations(results, out_path)

print("\n" + "=" * 70)
print("ANALYSIS")
print("=" * 70)

analysis = analyze_results(results)

print(f"\nTotal Samples: {analysis.total_samples}")
print(f"C-Score Sensitivity Distribution:")
print(f"  Sensitive: {analysis.c_score_sensitive_count}")
print(f"  Not Sensitive: {analysis.c_score_not_sensitive_count}")

print(f"\n{'='*70}")
print("FEATURE IMPORTANCE (by correlation with C-Score sensitivity)")
print("="*70)

for i, (name, corr) in enumerate(analysis.feature_importance[:15], 1):
    strength = "STRONG" if corr > 0.5 else "MODERATE" if corr > 0.3 else
"WEAK"
    bar = "âˆ^" * int(corr * 20)
    print(f"  {i:2}. {name:35} {corr:+.3f} {bar} ({strength})")

print(f"\nBest Predictors (|corr| > 0.3): {'',
'.join(analysis.best_predictors) or 'None'}")

print(f"\nSuggested Thresholds:")
print(f"  E-Score threshold:
{analysis.suggested_e_score_threshold:.2f}")
print(f"  Q-Score threshold:
{analysis.suggested_q_score_threshold:.2f}")
print(f"  Combined threshold:
{analysis.suggested_combined_threshold:.2f}")

# Save analysis
analysis_data = asdict(analysis)
with open(out_path / "benchmark_analysis.json", "w") as f:
    json.dump(analysis_data, f, indent=2, default=str)
print(f"\nSaved analysis to {out_path / 'benchmark_analysis.json'}")

# Print detailed sample results
print("\n" + "=" * 70)
print("DETAILED SAMPLE RESULTS")
print("=" * 70)

for result in results:
    marker = "ðŸŒ™ SENSITIVE" if result.c_score_deemed_sensitive else
"ðŸŒ™ NOT SENSITIVE"

    print(f"\n{result.sample_name}:")
    print(f"  Scores: E={result.e_score:.2f}, Q={result.q_score:.2f},
Sum={result.e_q_score_sum:.2f}")
    print(f"  Entities: total={result.total_entity_count},
types={result.unique_entity_type_count}, density={result.entity_density:.2f}")
    print(f"  Entity Types: {result.e_score_entity_types}")
    print(f"  Flags: person={result.has_person},
location={result.has_location}, high_risk_combo={result.has_high_risk_combo}")
    print(f"  Mask: ratio={result.mask_ratio:.2f},
tokens={result.mask_token_count}")
    print(f"  C-Score: {result.c_score_risk_level}
({result.c_score_risk_score:.2f}) -> {marker}")
    print(f"  C-Score Factors: {result.c_score_factors_detected}")

elif checkpoint.was_interrupted:

```

```

print("\n" + "=" * 70)
print("RUN INTERRUPTED")
print("=" * 70)
print(f"Progress saved: {len(results)}/{len(samples)} samples")
print(f"Resume by running the same command again.")
print("Use --fresh to start over from scratch.")

```

```

print("\n" + "=" * 70)
print(f"Output directory: {out_path}")
print("Files:")
print(" - checkpoint.json (resumable progress)")
print(" - pipeline_results.json (all results)")
if not checkpoint.was_interrupted and results:
    print(" - benchmark_analysis.json (analysis)")
    print(" - *.png (visualizations)")
print("=" * 70)

```

```

    return analysis if not checkpoint.was_interrupted and results else None,
results

```

```

# =====
# CLI Entry Point
# =====

```

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Benchmark C-Score necessity with checkpoint support",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""

```

Examples:

```

# Run benchmark (will resume from checkpoint if exists)
python benchmark.py example_inputs.txt

```

```

# Force fresh start (ignore checkpoint)
python benchmark.py example_inputs.txt --fresh

```

```

# Limit to first 10 samples
python benchmark.py example_inputs.txt -n 10

```

```

# Custom output directory and threshold
python benchmark.py example_inputs.txt -o results/ -t 0.4

```

Checkpoint Behavior:

- Progress is saved after each sample
- If interrupted (Ctrl+C), run again to resume
- Checkpoint invalidated if input file changes
- Use --fresh to force restart

```

"""
)
parser.add_argument(
    "input_file",
    type=str,
    help="Path to the input text file containing benchmark samples"
)
parser.add_argument(
    "-o", "--output-dir",
    type=str,
    default="benchmark_outputs",
    help="Directory to save output files (default: benchmark_outputs)"
)
parser.add_argument(
    "-n", "--num-samples",
    type=int,
    default=None,

```

```

        help="Maximum number of samples to process (default: all)"
    )
    parser.add_argument(
        "-t", "--threshold",
        type=float,
        default=0.3,
        help="C-Score risk threshold for sensitivity (default: 0.3)"
    )
    parser.add_argument(
        "--fresh",
        action="store_true",
        help="Ignore existing checkpoint and start fresh"
    )

args = parser.parse_args()

run_benchmark(
    input_file=args.input_file,
    output_dir=args.output_dir,
    max_samples=args.num_samples,
    c_score_threshold=args.threshold,
    fresh_start=args.fresh
)

```